

Project Proposal

Project members:

Sanabil Mustafa 24k-1014

Ayesha Tariq 24k-0785

Title

Nano Table | Data Management System

1. Introduction

In modern applications, users often need flexible tools to manage different types of structured data such as habits, books, attendance, or expenses. Most systems are rigid and designed for a single purpose.

This project aims to develop a **dynamic table-based system** where users can:

- Create custom tables
- Define their own columns (fields)
- Store and manage data according to their needs

The system acts as a **mini customizable database interface**, similar to a simplified version of Notion's table feature.

2. Problem Statement

Existing systems like habit trackers, attendance systems, or book managers are **separate applications with fixed schemas**.

Problem:

- Lack of flexibility
- Data cannot be reused across contexts
- Users must switch between multiple tools

Solution:

A unified system where users can create **custom table structures dynamically** and use them for multiple purposes.

3. Methodology / Workflow

1. User creates a new table (e.g., "Habit Tracker")
 2. User defines columns:
 - Column name (e.g., "Habit Name")
 - Data type (Text, Number, Date, Boolean)
 3. System stores schema in database
 4. User inserts records (rows)
 5. User can:
 - View data
 - Filter data
 - Update/delete records
-

4. Application Architecture



3-Tier Architecture:

- **Presentation Layer (GUI)**
 - **Application Layer (Logic handling)**
 - **Database Layer (Storage)**
-



Database Design (Core Idea)

Tables:

- **Users**
- **Tables** (user-created tables)
- **Columns** (stores column metadata)
- **Rows**
- **CellValues**



This is the **key highlight** of your DBMS part.

5. Tools & Techniques

- Language: Python
 - Database: MySQL / SQLite
 - Concepts used:
 - ER Modeling
 - Normalization
 - OOP (for SDA)
 - SQL Queries (joins, filters)
-

6. GUI Design (Simple but clean)

Screens:

1. Dashboard → List of tables
2. Table Creator → Add columns
3. Table View → Show rows
4. Add Record Form

👉 Keep GUI simple but structured (marks are for design, not beauty)

7. Functional Requirements

- Create new table
 - Add custom columns
 - Insert records
 - View records
 - Filter records
 - Delete/update records
-

Non-Functional Requirements

- Usability: Simple interface
- Performance: Fast query execution
- Scalability: Supports multiple tables

- Reliability: Data consistency maintained
-

8. Testing + Test Cases

Example:

Test Case	Input	Expected Output
Create Table	Name + columns	Table created
Add Record	Valid data	Row inserted
Invalid Input	Wrong data type	Error message

9. External Libraries

- Flask
-